



Mahidol University
Faculty of Information
and Communication Technology



**MediScan: A System for Managing and Deploying Machine Learning
Models as a Service for Medical Image Analysis**

BY

MR. THANAPHAT SUMRITH 6488092
MR. PUNNARUJ WUTHIJARIYASAK 6488104
MR. PUWADON WUTTISAK 6488114

ADVISOR

Associate Professor Dr. Suppawong Tuarob

**A Senior Project Submitted in Partial Fulfillment of
the Requirements for**

**THE DEGREE OF BACHELOR OF SCIENCE
(INFORMATION AND COMMUNICATION TECHNOLOGY)**

**Faculty of Information and Communication Technology
Mahidol University
2024**

COPYRIGHT OF MAHIDOL UNIVERSITY

ACKNOWLEDGEMENTS

To my senior project adviser, Assoc. Prof. Dr. Suppawong Tuarob, I would like to extend my sincere gratitude for all their help, support, and encouragement during the project. Their knowledge and perceptions have been invaluable in guiding me through the difficulties and ensuring my success.

We would especially like to thank the members of the project committee for their insightful comments and helpful criticism, which have greatly improved this effort. I am also incredibly grateful to my fellow participants and collaborators for their invaluable efforts and collaboration, which enabled me to accomplish the project's goals.

Finally, I would like to express my sincere gratitude to the faculty members of Mahidol University's Faculty of Information and Communication Technology for their commitment and knowledge sharing, both of which have influenced my academic growth. I also want to thank my family and friends for their patience, encouragement, and continuous support during this journey.

MR. THANAPHAT SUMRITH

MR. PUNNARUJ WUTHIJARIYASAK

MR. PUWADON WUTTISAK

**MEDISCAN: A SYSTEM FOR MANAGING AND DEPLOYING MACHINE LEARNING MODELS
AS A SERVICE FOR MEDICAL IMAGE ANALYSIS**

MR. THANAPHAT SUMRITH 6488092 ITCS/B

MR. PUNNARUJ WUTHIJARIYASAK 6488104 ITCS/B

MR. PUWADON WUTTISAK 6488114 ITCS/B

B.Sc. (INFORMATION AND COMMUNICATION TECHNOLOGY)

PROJECT ADVISOR: ASSOCIATE PROFESSOR DR. SUPPAWONG TUAROB

ABSTRACT

Recent advancements in medical imaging and diagnostics have shown that the requirement of flexible machine learning models, that can be fine-tuned and personalized, is possible. This work presents a machine learning model management system, developed for the medical domain which is used to manage a machine learning model and deploy the machine learning model onto the Mobile. Doctors can build, edit, and assess machine learning models on specific medical datasets through the system. In addition, doctors can label datasets in real-time to further improve the model's performance. This backend allows doctors to operate on various models in parallel and allows patients to interact with the system for diagnostic purposes only, but they are not allowed to change or label data. On the system, there is a suite of API endpoints for image uploading, getting images, creating project, approving modifying, labeling, model evaluation, and deployment. This project provides a platform to create medical diagnostic models tailored to medical professionals and machine learning alongside the medical professionals.

KEYWORDS :

40 P.

เมดิคสแกน: ระบบสำหรับการจัดการและปรับใช้โมเดลการเรียนรู้ของเครื่องในรูปแบบบริการสำหรับการวิเคราะห์ภาพทางการแพทย์

MR. THANAPHAT SUMRITH ITCS/B
MR. PUNNARUJ WUTHIJARIYASAK 6488104 ITCS/B
MR. PUWADON WUTTISAK 6488114 ITCS/B

วท.บ. (เทคโนโลยีสารสนเทศและการสื่อสาร)

อาจารย์ที่ปรึกษาโครงการ: รศ. ดร. ศุภวงศ์ ทวีรอบ

บทคัดย่อ

ความก้าวหน้าล่าสุดในด้านการถ่ายภาพทางการแพทย์และการวินิจฉัยโรคแสดงให้เห็นว่าความต้องการโมเดลแมชชีนเลิร์นนิงที่มีความยืดหยุ่น ซึ่งสามารถปรับแต่งและปรับให้เหมาะสมกับการใช้งานส่วนบุคคลนั้นเป็นไปได้ งานวิจัยนี้ได้นำเสนอระบบการจัดการโมเดลแมชชีนเลิร์นนิงที่พัฒนาขึ้นสำหรับวงการแพทย์ ซึ่งใช้สำหรับจัดการโมเดลแมชชีนเลิร์นนิงและปรับใช้โมเดลดังกล่าวบนอุปกรณ์มือถือ แพทย์สามารถสร้าง แก๊ซ และประเมินโมเดลแมชชีนเลิร์นนิงบนชุดข้อมูลทางการแพทย์เฉพาะทางผ่านระบบนี้ นอกจากนี้ แพทย์ยังสามารถติดป้ายกำกับข้อมูลชุดข้อมูลแบบเรียลไทม์เพื่อปรับปรุงประสิทธิภาพของโมเดลให้ดียิ่งขึ้น

ระบบ Backend นี้ช่วยให้แพทย์สามารถทำงานกับโมเดลหลายโมเดลพร้อมกันได้ และช่วยให้ผู้ป่วยสามารถโต้ตอบกับระบบเพื่อวัตถุประสงค์ในการวินิจฉัยเท่านั้น โดยไม่ได้รับอนุญาตให้เปลี่ยนแปลงหรือติดป้ายกำกับข้อมูล ระบบยังมีชุด API สำหรับการอัปโหลดรูปภาพ ดึงรูปภาพ การสร้างโปรเจกต์ การอนุมัติ การแก๊ซ การติดป้ายกำกับ การประเมินโมเดล และการปรับใช้โมเดล โปรเจกต์นี้มอบแพลตฟอร์มสำหรับการสร้างโมเดลวินิจฉัยทางการแพทย์ที่ปรับแต่งให้เหมาะสมสำหรับบุคลากรทางการแพทย์และแมชชีนเลิร์นนิงที่ทำงานร่วมกับบุคลากรทางการแพทย์อย่างใกล้ชิด

CONTENTS

	Page
ACKNOWLEDGEMENTS	ii
ABSTRACT	iii
Introduction.....	1
1.1 Motivation.....	1
1.2 Problem Statement.....	2
1.3 Objectives of the Project.....	2
1.4 Scope of the Project.....	3
1.5 Expected Benefits.....	3
Background.....	4
2.1 Understanding Model Management in Machine Learning.....	4
2.2 Machine Learning Architectures for Medical Image Classification.....	5
2.3 Importance of Data Annotation and Labeling in Medical Machine Learning.....	5
2.4 The Importance of Real-Time Diagnostics and Early Detection.....	6
2.5 Existing Technologies for Machine Learning Model Management.....	6
Related works.....	7
3.1 Machine Learning Model Management.....	8
3.2 Machine Learning in Medical Image Classification.....	9
3.3 Skin Cancer Classification using 3 Machine Learning.....	10
3.4 Comparative Analysis of Existing Tools and Challenges in Machine Learning Model Management.....	11
3.5 Challenges in Machine Learning Model Management.....	12
Methodology.....	13
4.1 System Overview of MediScan.....	13
4.2 Architecture Overview.....	15
4.3 Functional Decomposition.....	16
4.4 Use Case Diagram.....	19
4.5 Data Flow Diagram.....	22
4.6 Database Design.....	25
4.7 Machine Learning Methodology.....	27
4.8 Security Measures.....	28
evaluation and results.....	29
5.1 Model Testing.....	29
5.1.1 Objective and Model Testing.....	29

5.1.2 Testing Methodology.....	30
5.1.3 Definition of Success Criteria.....	31
5.1.4 Model Evaluation.....	32
Conclusion and Discussion.....	40
6.2.1 Future work.....	40
6.2.2 Expand the Dataset size.....	40
6.2.3 Improve Model Performance.....	41
6.2.4 Develop a User Interface.....	41
6.2.5 Limitations.....	41
6.2.6 Ethical Considerations.....	42
REFERENCES.....	43
APPENDIX A.....	43

CHAPTER 1

INTRODUCTION

This chapter outlines 5 sections: Motivation, Problem Statement, Objectives of the Project, Scope of the Project, and Expected Benefits. Each section provides an understanding of the project purpose.

1.1 Motivation

Healthcare is still a huge barrier for people living in remote or under served areas. In these areas many people, not realizing that they are sick, notice concerning symptoms such as unusual skin lesions, but have no nearby medical facilities or people trained to diagnose them. The early detection of one of the most common and potentially deadly diseases — skin cancer — is critical for helping patients outcomes. Nevertheless, diagnosis is frequently delayed or missed in these settings because of lack of expertise, tools necessary for accurate diagnosis.

Not only did patients face these challenges, many of the advanced diagnostic technologies such as machine learning that could help improve the experience of health care professionals also present difficulties because they lack integration into practice. It is true that machine learning can speed up diagnostic accuracy, but many such tools were not made with healthcare in mind. As doctors, we want systems that are easy to use, secure, adaptable to our particular diagnostic workflows, and they have to check out thoroughly in front of us. Without these tools, the gap for practical application of advanced technologies grows ever larger, leaving patients and professionals still underserved. Removing these barriers is critical to the achievement of improved access to timely and accurate medical diagnostic targeting health care resources that are itself limited.

1.2 Problem Statement

Despite the existence of accessible skin cancer diagnostics, no such access has been achieved, particularly in rural and underserved areas where medical specialists are lacking. Every year, millions of cases of skin cancer are diagnosed. However, many patients are delayed or misdiagnosed because there are no accessible diagnostic tools and specialized dermatological care available. To overcome this gap, machine learning has enormous promise, and existing systems typically don't deliver. However, current solutions for model management and deployment are not designed to meet the specific needs of healthcare. They fail to address critical issues such as:

- To handle sensitive information of a medical nature Data privacy and security.
- And easy to use for non technical users like doctors and healthcare professionals.
- Immediate decision making, real time diagnostics.
- The ability to continuously adapt to the ever changing characteristics of medical data and patient demographics.

Machine learning presents exciting opportunities to healthcare professionals; however, these limitations prevent them from using machine learning to its full potential, locking this life saving diagnostics out of reach for many. This will continue to result in countless patients meeting unnecessary delays, endangering patients and their families with late stage diagnoses and preventable deaths without a solution.

1.3 Objectives of the Project

The objectives of this project are as follows:

- **Model Management:** It allows doctors to create and manage several machine learning models for medical imaging datasets.
- **Dataset Labeling:** Last, it allows doctors to label datasets in order to improve model accuracy while restricting patients from labeling data.
- **Model Evaluation:** You can offer a feature to doctors for the verification of machine learning models and selecting the best model that can be used for their clinics.

- **Flexible Model Deployment:** It allows doctors to deploy particular models, or different versions of models, for use in medical diagnostics.
- **API Integration:** Unfortunately, no such complete set of API endpoints exists that handles uploading images, managing datasets, labeling data, evaluating models and deploying models for prediction tasks.

1.4 Scope of the Project

- **Backend Development:** Backend system for machine learning model management, handling dataset, user roles etc.
- **User Roles:** Adjustment of the model and data to generate user specific results for doctors and standard clinical inference using the same model for patients; differentiation between doctors (model and data creators, labelers, and reviewers) and patients (users of the system only for diagnostic predictions).
- **Model Types:** It will support different types of machine learning models such as image classification and detection ones used in medical datasets.
- **API Endpoints:** The various functions involved in handling image uploading, model evaluation and model deployment will be developed into a set of APIs.

1.5 Expected Benefits

- **Improved Diagnostic Tools:** It allows doctors to tune machine learning models to local data and to increase diagnostic modality accuracy.
- **Flexible Model Management:** Give medical professionals the ability to build and deploy as many models, from one through dozens yet, depending on the task at hand.
- **Secure Data Handling:** Require that datasets be labeled by only authorized personnel and patients can access the system for diagnostic purposes without touching the data.

CHAPTER 2

BACKGROUND

This chapter features foundational concepts as key to the MediScan system, namely, how machine learning can be used in medical diagnostics, the limitations of model management in the healthcare sector as well as which machine learning architectures are suitable for medical image classification. It also shows that real time diagnosis and early diagnosis can ameliorate patients outcomes.

2.1 Understanding Model Management in Machine Learning

Model management is the lifecycle processes of developing, training, deploying, maintaining and optimizing machine learning models. Patient outcome predictions in the healthcare environment (e.g. patient outcome prediction) is critically dependent on accurate and reliable predictions; that the ability to predict patient outcomes is manageable.

MLflow and ModelDB are general purpose tools for version tracking, performance and deployment monitoring. However these tools are not designed just for healthcare but have to be modified for healthcare needs.

Healthcare-specific model management systems enable user to:

- Monitor and compare model performance metrics.
- Model for specific datasets for medical impact.
- Facilitate real time predictions in medical workflows.

In addition, the MediScan project offers a model management system that extends from existing tools to streamline dataset labels, model train and deploy. Therefore, doctors can manipulate their models, but currently still maintain a malleable system against dynamic diagnostic requirements.

2.2 Machine Learning Architectures for Medical Image Classification

Several machine learning architectures have been adapted to medical imaging because of their accuracy and efficiency. These architectures presented are YOLO (You Only Look Once), Each model brings unique capabilities to medical diagnostics:

YOLO (You Only Look Once): The individual of this method is well known for its real time object detection capability which performs object localization and classification at the same time in a unified task in a single pass. And in terms of speed and efficiency, we've found it to be really really great for applications where you really don't have a lot of time: say, you have some large dataset and suddenly there's something, something went wrong.

With these architectures, MediScan merges flexible tools suited to required diagnostic requirements. This allows the selection of a model that best meets the clinical needs of a doctor, such that the diagnosis is as precise in its definition as it is speed in its time of consideration.

2.3 Importance of Data Annotation and Labeling in Medical Machine Learning

Labeled data is a key ingredient for supervised machine learning: It allows us to train our algorithms. Additionally, this process frequently must take place in medical settings, and the data expert is also the person who most accurately annotates the condition, such as skin cancer. Medical datasets are slow in being labeled, however the use of them demands reliable infrastructures to ensure the accuracy and consistency of labeling.

But with MediScan, doctors are provided with a secure labeling interface to do this quickly. With integrated labeling and model management, this ensures that we have trained and fine tuned datasets so that they are ready to train, but with high quality standards. Similarly, as a means to enable streamlining of the annotation process that allows for reliable clinically relevant machine learning models, mediScan offers a method.

2.4 The Importance of Real-Time Diagnostics and Early Detection

ML powered real time diagnostics is a game changer for healthcare. Instantly processing and analyzing medical data can make faster decisions in something as important as detecting life threatening conditions or triage patients. For example, in dermatology, images could be analyzed in real time for doctors to identify a malignant lesion during the consultation while reducing time between a diagnosis and treatment.

Detection of disease at an earlier time point in the disease continuum through use of ML models leads to better patient outcomes as a result of intervention at an earlier time point in the course of disease progression. In fact, early stage treatments are cheaper overall—at a reduced cost and increased effectiveness—mostly due to their being less invasive.

What MediScan is all about, is being able to give the doctors fast, accurate, actionable insights. It brings together the latest ML architectures and easy to use tools so that healthcare professionals can provide timely and effective care.

2.5 Existing Technologies for Machine Learning Model Management

With increasing reliance on ML model tracking, version control and evaluation by industries such as finance and ecommerce, model management systems have become more important. There are such types of these platforms as MLflow and Model DB, which offer methods for tracking, versioning, deploying models. However, these platforms cannot be used for medical diagnostics because they cannot do unique medical specific annotation processes, they lack the ability to securely process the patient data and they do not use their own evaluation metrics for health.

CHAPTER 3

RELATED WORKS

In this chapter we review previous work on managing machine learning (ML) models, with special reference to those used in medical diagnostics. In this paper, it analyzes and identifies weak and strong features of present model management systems. This chapter also looks at current image classification models for health care, discussing the promising diagnostic accuracy as well as remaining problems in this area. Finally, the paper discusses key practical issues including handling of medical data, validation of models, ensuring regulatory compliance, and bringing ML systems into the healthcare workflow.

Recently, machine learning (ML) has been extensively used in medical diagnostics, including medical imaging for diseases such as skin cancer, and a variety of malignancies. For healthcare providers to be able to optimize and deploy the best performing models for clinical use, we believe effective model management for ML in clinical contexts is necessary. Following this introduction, this section presents existing approaches and tools for ML model management focusing on the field of healthcare, then introduces advanced image classification models that are commonly used in medical diagnosis.

3.1 Machine Learning Model Management

For environments with large numbers of experiments (and retraining and deployment), model management systems are important in machine learning. Tracking multiple versions of models, and monitoring their performance, these systems make it possible to deploy them as per situational requirements. Patient outcome relies on model accuracy and reliability so model management in the medical field should strictly follow the same rules as in other fields. Despite the fact that general model management frameworks such as MLflow and ModelDB offer generic model management capabilities [1], [2] that users can use to efficiently log, store, version their ML models, these are not designed specifically for healthcare [1].

TensorFlow Extended (TFX) has an ML deployment workflow supported in another platform and a Model Analysis library included. But it has no tools for incorporating medical datasets or compliance with regulatory requirements that are vital in health care [3]. Kubeflow, another popular choice for ML pipelines in Kubernetes, has support for model versioning along with deployment, yet managing it in real time clinical environments requires heavy technical skills.

Furthermore, the process suggested for evaluation of machine learning models in healthcare applications by Dawson and Sato (2021) is also added. However, their study also stresses the dimension of lifecycle management: continuous monitoring, evaluation and adaptation of models towards real world data [5]. The proposed process addresses noise, bias, and evolving medical data and is therefore particularly suited to the healthcare problem domain. The results of this study emphasize the need for systems that support the complexities of deploying ML models in clinical environments.

3.2 Machine Learning in Medical Image Classification

With the advancement of deep learning and convolutional neural networks (CNNs), medical image classification has become important. In fact, image detection tasks are well handled by a CNN based model called YOLO (You Only Look Once) [6], that can generate fast real time predictions through just a single pass on the images. Variants of YOLO, including YOLOv4 and YOLOv5, have been used to detect skin cancer owing to their high accuracy and speed [7]. YOLOv8 also improves model accuracy and inference time, making it even better suited for clinical applications such as real time diagnostics [8].

Other medical imaging tasks have been implemented with other ML models, like DETR (Detection Transformer). DETR utilizes a transformer based architecture that better processes the spatial relationships in images. This is particularly useful in analyzing complex medical images, as occurs in, for example, radiology [9]. For example, we demonstrate that DETR can be used to detect anomalies in chest X-rays and mammograms, demonstrating a promising avenue for aiding radiologists with complex image data [10].

Other models also on EfficientNet architectures are EfficientDet which have also shown good performance in medical image classification, especially for limited computing environment applications. Computationally EfficientDet reduces the computation time, for environments where computation is a constraint, for rural or under-resourced healthcare facilities [11].

3.3 Skin Cancer Classification using 3 Machine Learning

In regions of high UV exposure, skin cancer classification is more difficult because of variability in lesion appearance and environmental factors. Recently, widely successful deep learning based image classification models have been evaluated for their ability to detect skin lesions with diagnostically accuracy on par with dermatologists. ResNet and InceptionV3 two popular CNN architectures are studied classifying for benign and malignant lesions and the models are able to have high sensitivity and specificity over curated datasets [12].

Transfer learning has also become a standard in skin cancer detection, where a model trained on a large, general image dataset is fine tuned with medical image data. As shown in a study where a ResNet model was fine tuned on the HAM10000 dataset, this technique significantly improves the accuracy of skin lesion classification above 85 % for malignant / benign classification.

This project attempts to solve the complexities of using ML in medical diagnostics by building a model management system for healthcare providers. In the application of machine learning in clinical use, the system will enable healthcare professionals to manage and optimize ML models for specific medical needs and appropriate usage of machine learning in clinical use.

3.4 Comparative Analysis of Existing Tools and Challenges in Machine Learning Model Management

→ COMPARISON CHART

FEATURE / APPNAME	MLFLOW	MODELDB	KUBEFLOW	MEDISCAN
MODEL VERSIONING	✓	✓	✓	✓
EXPERIMENT TRACKING	✓	✓	✓	✓
REAL-TIME DEPLOYMENT	✗	✗	✓	✓
CUSTOM MODEL CREATION	✗	✗	✗	✓
DATA LABELING INTEGRATION	✗	✗	✗	✓
DOMAIN-SPECIFIC FOCUS	General-Purpose	General-Purpose	General-Purpose	Customized to medical dianostics
SUPPORT FOR MEDICAL APPLICATIONS	Limited	No	Limited	Designed for healthcare
EASE OF USE FOR NON-TECHNICAL USERS	Moderate	Low	Low	High

Figure 3.1 Comparison chart

3.5 Challenges in Machine Learning Model Management

Despite advancements in ML model management, several challenges persist, particularly in healthcare applications:

Data Quality and Annotation: Medical datasets are often expertly annotated, which is very time consuming and labor intensive work. This is also a critical bottleneck [11], [12] to make sure the labels of a large dataset agree and are accurate.

Model Interpretability: When health care applications come into play especially if clinicians are not going to trust these predictions, this becomes very vital. This usually doesn't like using complex models like deep neural networks [6].

Adapting to Changing Data: The medical data is evolving over time because demographics of patient populations change, diagnostic tools and treatment methods change. Accuracy requires that models be continuously updated and retrained [4].

Integration with Clinical Workflows: Integration has more depth with customizing the ML models and carefully planned integration to existing clinical workflows. But to be practical, it must be able to work with electronic health record systems as well as other healthcare software [7], [8].

Limited Accessibility for Non-Technical Users: However, many existing tools need a good dose of technical expertise to manage and deploy ML models. Accessible solutions to the needs of non technical healthcare professionals e.g. doctors and nurses [1], [2]. In order for ML to actually get useful in healthcare, the solutions have to be simpler and easier to use. These tools should allow healthcare workers to use them without having to know complex algorithms, or even programming at all. Users could be benefited a lot by user friendly interfaces that provide easy to follow clear insights or visualizations. That would free up doctors and nurses to focus on patient care based on model predictions, rather than to be experts in the technology.

CHAPTER 4

METHODOLOGY

4.1 System Overview of MediScan

To address the long-standing challenges in medical workflows, MediScan aims to solve these problems with the addition of powerful machine learning models, the most robust database and backend in the world. It simplifies diagnostic workflows to achieve high accuracy at the same time ensuring data security. With these features usable by doctors as well as patients, it supports integration with user-friendly interfaces.

Task management and communication is taken care of by a backend built on top of a TCP server with Redis deployed on it for efficient communication. It is responsible for keeping the data processes (image classification, feedback collection, model evaluation, metadata extraction) Metadata, user information, logs, machine learning model configurations are all stored using a MySQL database for efficient queries and data integrity without duplicating.

When preprocessing the datasets, the diagnostic images are resized, normalized as well as augmented to improve the datasets. We set out to develop these high quality, doctor labeled datasets that are trained for machine learning models at scale with a meaningful clinical purpose.

Local operations with the modular machine learning models (YOLO) run inside the system. With this setup, doctors are able to rapidly predict and fine tune models that are capable of meeting their diagnostic needs. Traceability and accountability for all system actions including model update and image upload are logged.

MediScan architecture integrates all components within the whole system for security, efficiency and adaptability for various healthcare needs.

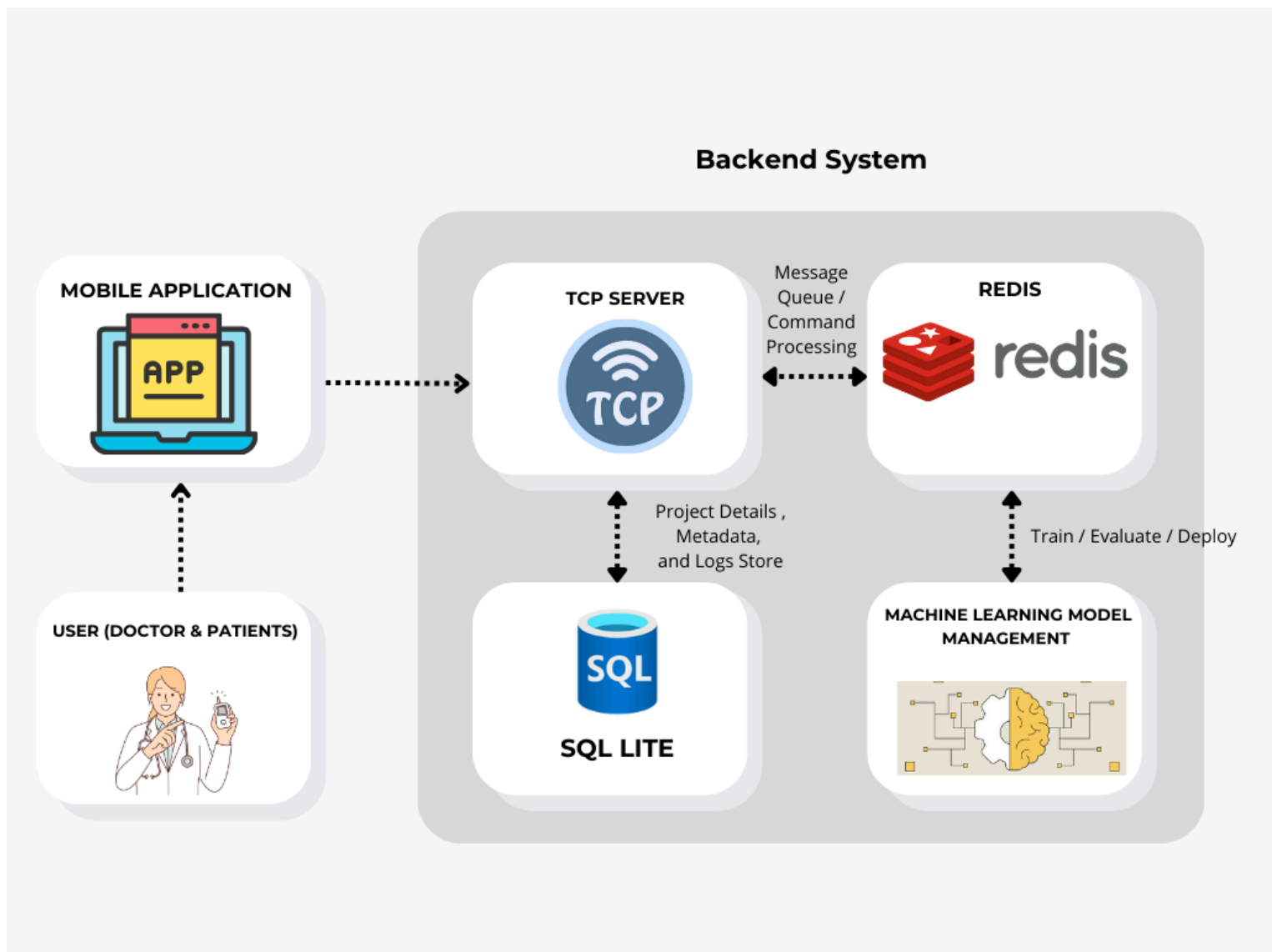


Figure 4.1: MediScan System Overview

4.2 Architecture Overview

By carefully designing the MediScan architecture, it will facilitate the low cost, efficient image classification, robust model management and smooth deployment in medical diagnosis. This part describes the system and specifies the technical parts and the relation between them.

Backend

The backend, implemented using a TCP server and Redis, as the central hub of the system, handling:

- Data Processing: Preprocessing Image and metadata pipelines are handled by it.
- Task Management: Takes advantage of Redis with respect to task handling (i.e. model training, evaluation, predictions and deployments).
- Integration with ML Libraries: The APIs used include those of TensorFlow, PyTorch and YOLO for training and deployment of the model.

Machine Learning Models

The MediScan system leverages three pre-trained models YOLOv8 to cater to diverse diagnostic requirements:

- YOLOv8: Fastest for high speed image detection and ideal for applications needing high speed analysis.

Database

MySQL is used as the relational database to store and manage structured data, including:

- Project Metadata: Names and descriptions of diagnostic projects.
- User Profiles: Information for doctor and patient including roles and permissions.
- Diagnostic Images: Data extracted during preprocessing: file paths, labels, metadata.
- Operational Logs: Ability to record user actions and system activities to keep an account.

DataPreprocessing, Data Collection:

- Data Collection: Images from clinical cases are obtained as diagnostic images and labeled by doctors through a secure interface. Datasets created in this way are relevant and aligned with real world medical scenarios.
- Preprocessing Pipeline: Resizing, normalization, data augmentation, metadata extraction wraps included.

Logging and Monitoring:

The logging and monitoring system provides:

- Traceability: It tracks every action including transferring images, training and testing the model, and even makes predictions.
- Detailed Insights: Also provides auditing and performance reviews, a continuous system improvement feature.

4.3 Functional Decomposition

The MediScan system is designed with modular components that hierarchically decompose the system functionality to achieve a well-defined design process and workflow.

Functional Decomposition Diagram Description:

MediScan operations are decomposed into distinct, hierarchical functions in the functional decomposition diagram. The MediScan Backend System joins various work flows around management of machine learning based diagnostics. At the core, doctors can make projects, link machine learning models and manage diagnostic datasets. The patients upload diagnostic imaging which are enriched with metadata and stored in a secure database. The doctors are then used to label the images to create high quality datasets the models can be trained on. In addition, they can make edits to or approve labels on images so that data is accurate.

It is a system that supports advanced functionalities of training machine learning models from labeled data and deployment of trained models for real time diagnostic application. Metrics such as accuracy, precision, and recall are used to evaluate doctors' performance with that model, and they pick up the best performing model to deploy. Feedback mechanisms enable doctors to refine datasets, and either

improve model training, or make it easier for a model to pull meaningful information from feedback responses. The deployed models make predictions which are accessible to both doctors and patients to help them make informed decisions. In addition, all past projects, predictions, and feedback logs are kept by the system. The combined confirmatory and diagnostic workflow is further complemented by its comprehensive functional framework.

Image Management:

- Diagnostic images are enriched with metadata, uploaded by doctors and patients, and stored securely. Doctors can therefore modify and approve image labels thereby leading to high quality datasets.

Model Management:

- Machine learning models are trained by doctors, evaluated and the most accurate versions deployed for real time use.

Prediction and Feedback:

- Doctors validate the results that are generated by patients uploading images for classification. It provides us with feedback to refine the models, improve future predictions.

History and Auditing:

- It provides both traceability and transparency, because both doctors and patients can get access to historical data: project logs, past predictions, and feedback.

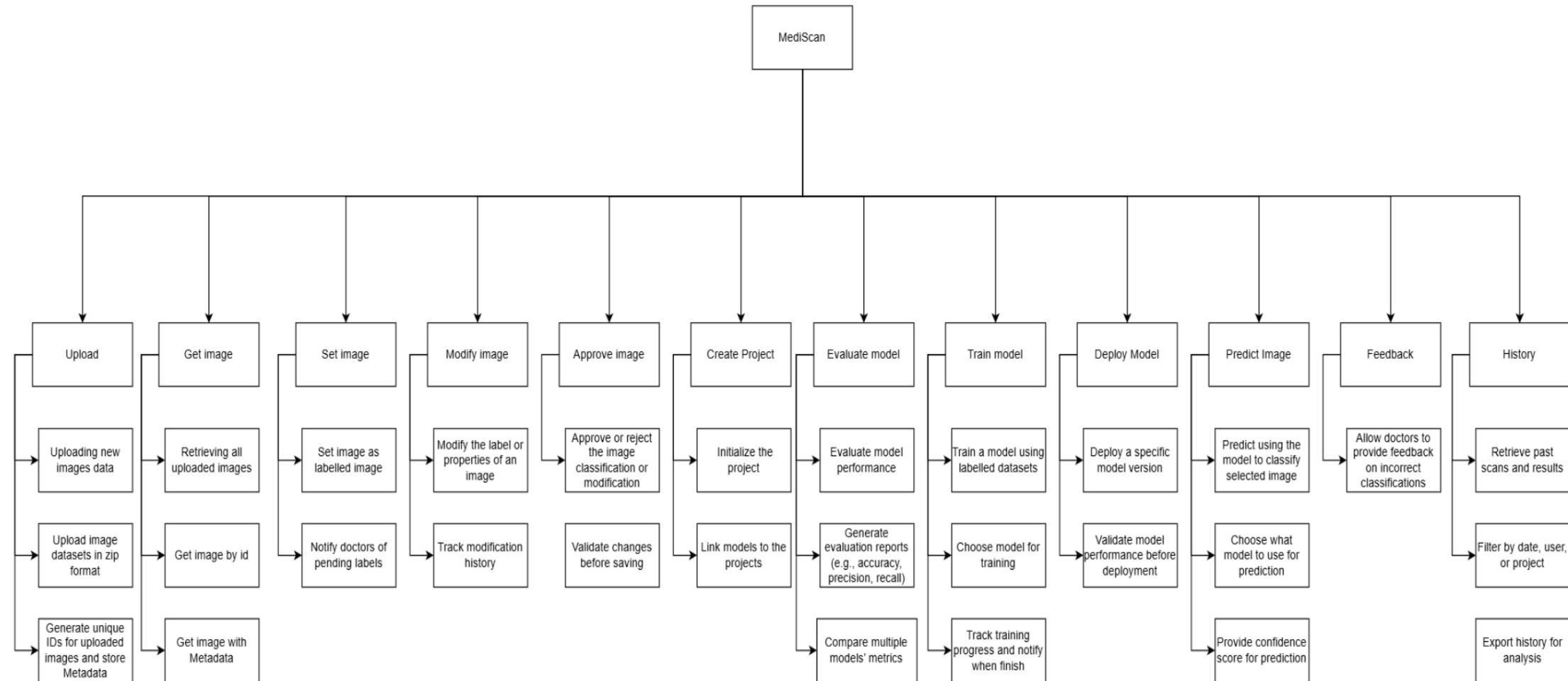


Figure 4.2: Functional Decomposition Diagram

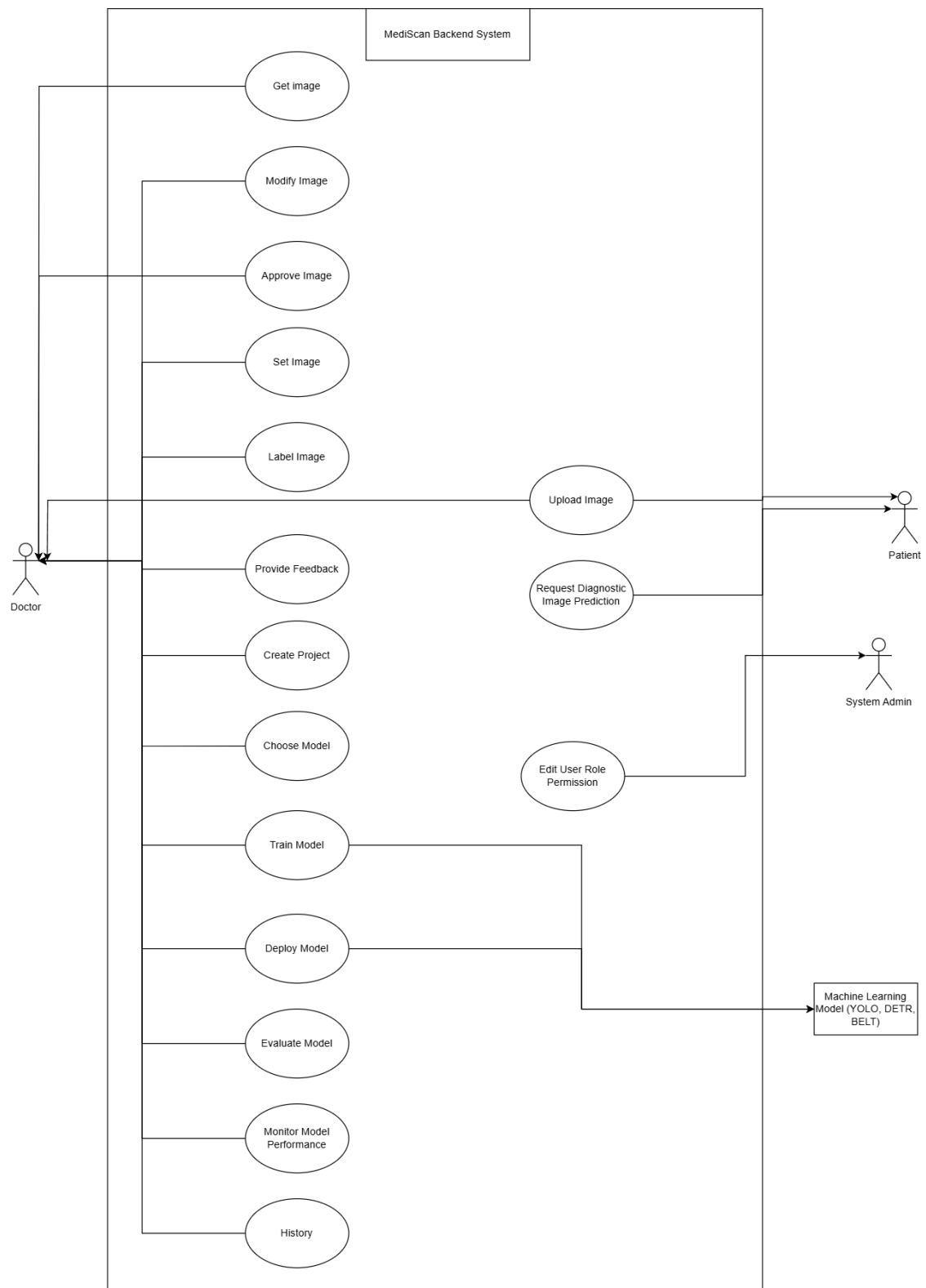
4.4 Use Case Diagram

In use case diagrams are shown how actors (doctors, administrators, patients) interact with a system. Use cases that come to mind include image upload, labeling, model training as well as generation of predictions.

Use Case Diagram Description

1. **Register** – To create an account users must use their email address, password and personal details. That includes agreeing to the terms and conditions of the MediScan system and includes agreeing to the terms and conditions of the MediScan system.
2. **Login** – This allows the user to authenticate its identity enabling the user access the system based on its role.
3. **Create Project** – Doctors can create Diagnostic projects, with name, description, and optional metadata. Each project supports a different set of diagnostic cases that live in the same workspace.
4. **Upload Image** – Patients can upload diagnostic images on a specific project. The metadata of the file path and the associated diagnostic labels are stored with each image.
5. **Set Image** – Doctors can modify an image to change the labeling or any other property of an image so that the image is accurate, saving it into a training dataset.
6. **Modify Image** – Doctors can edit labels or other properties of an image to ensure accuracy before it is included in a training dataset.
7. **Approve Image** – After modification or labeling, doctors can approve or reject images, to be included into certain projects or dataset.
8. **Choose Model** – Within projects, Doctors choose specific machine learning models like YOLO for training or prediction goals.
9. **Train Model** – Doctors can use labeled datasets associated with their projects and train the models they want using Train Model. The training process they use updates model performance for later use.

10. **Deploy Model** – Real Time Prediction can begin predictions in real-time. Deployment processes are managed by a doctor who selects suitable versions for diagnostic tasks.
11. **Predict Image** – Deployed model – Patients or Doctors can use this model to classify uploaded images. The prediction includes labels and the confidence scores from the system.
12. **Provide Feedback** – Doctors can be provided with feedback of wrong predictions or model behavior. The result of this feedback is to refine the model and datasets.
13. **Evaluate Model** – This page also has the Evaluate Model section for doctors where they can evaluate the trained models with accuracy, precision, recall, and F1-score metrics, to check that they are doing good performance.
14. **Monitor History** – Historical Data – All doctors as well as patients can see historical data, such as old projects, predictions, logs, and feedback. It is useful for reviewing how far we have come in terms of diagnosing and refining next operations.
15. **Edit Role Permission** – Change the Role Permission – System Admins manage the user's role and permission; i.e. that only the authorized user has access to the particular function of the MediScan system.
16. **Access Logs** – Doctors can log access to certain system outputs that are necessary to trace and document narratives. System Admins can also retrieve logs to see what happened during the system's operational history.

*Figure 4.3: Use Case Diagram*

4.5 Data Flow Diagram

The Data Flow Diagram (DFD) Level 0 (Figure 4.4) shows the overall data process in the MediScan system with major focus on the interactions between external entities, the back end and the database.

Data Flow Diagram Level 0 Description:

1. Inputs:

- Patients produce diagnostic images with their attributes.
- Doctors offer labeled datasets and start the model training and take part in the validation process of the predictions.
- System Admins are responsible for the users' roles and the usage of the systems.

2. Processes:

- Any transmitted images are analyzed, enhanced with relevant tags, and kept safe and sound.
- These datasets are used to train local models that do not compromise data privacy more so where the particular data belongs to a particular country only.
- The predictions are made based on models deployed and then provided to the users.

3. Outputs:

- Users can get diagnosis and feedback from the results displayed on the screens for both the doctors and the patients.
- This is for record keeping for audit and review.

4. Integration:

- The backend uses a MySQL database which is used in the storage as well as retrieval of information.
- Real time predictions for diagnostic tasks are made by trained models.

The Data Flow Diagram (DFD) at Level 0 describes the overall data processing and communication that is to be performed within the MediScan system. Doctors, Patients and A set of Machine Learning Models are the primary external actors of the system. Patients contribute through acting as data sources whereby they upload diagnostic images for classification. The captured images are then parsed by the MediScan Backend System to extract metadata from the images and save them in the database. Doctors interact with the system in multiple ways: they create projects, manage datasets, and kick off the model training process, among other tasks. The system enables the use of machine learning models as trainers and also feeds real-time image classifiers.

Once deployed, the model will process uploaded images to make predictions and return them back to the MediScan backend so that the predictions are shared with users. These predictions are verified by doctors and the feedback from doctors is used to refine future model prediction. All required data (projects, cases, images, model metadata and feedback) is stored in an SQLite database for efficient and secured data storing. The result of this circular data flow between the users, a machine learning model, backend, database is a highly integrated diagnostic system.

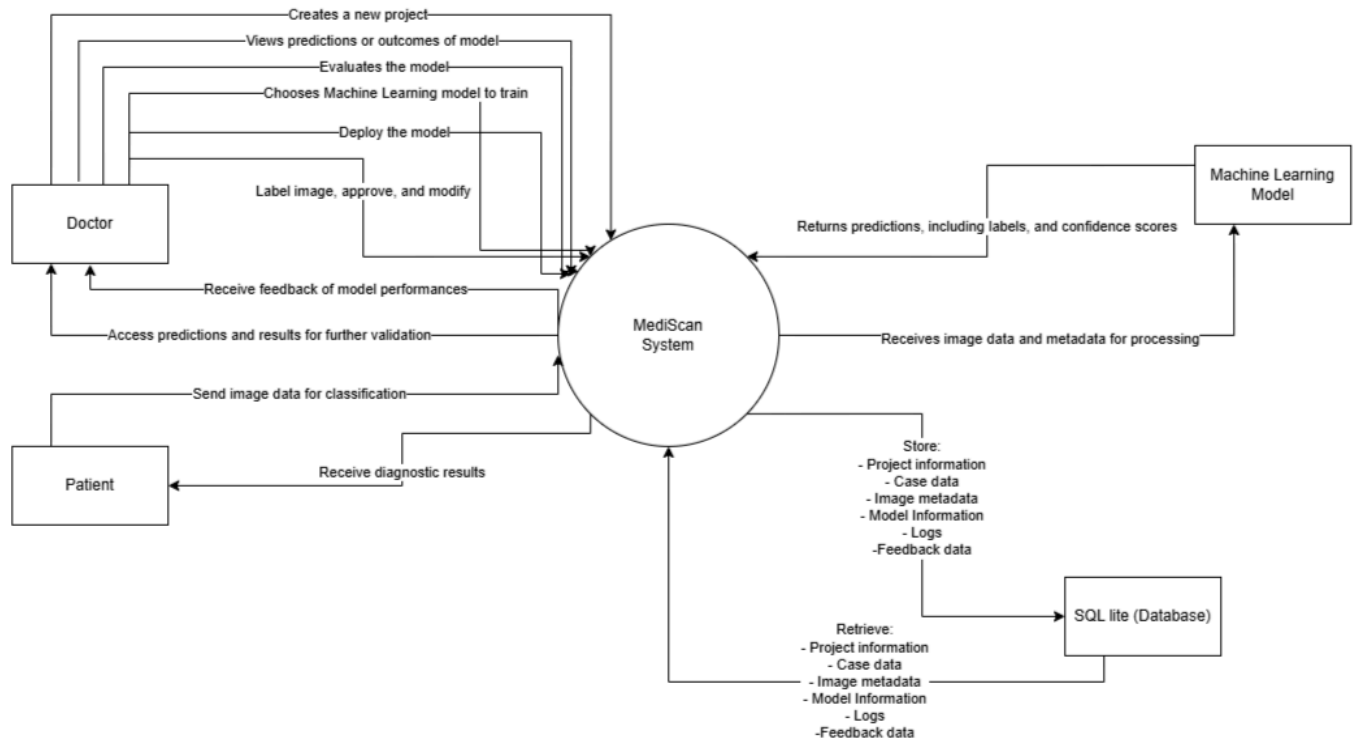


Figure 4.4: Data Flow Diagram Level 0

4.6 Database Design

The MediScan database schema, consisting of the Entity-Relationship (ER) diagram, is designed to manage diverse data flows and interactions in an efficient way.

In MediScan the structure and relations of the database and the entities in it are represented by the Entity – Relationship (ER) Diagram. All users information, which include roles (doctor or patient) is stored in Users entity including email and password. The user access levels within the system are indicated by this table. Doctors' led diagnostic projects (which correspond to projects) There is a name, description and time of creation for each project as a way to organize and manage diagnostic activities in a structured manner. Cases are subcategories of a given project that describe different diagnostic tasks for a given project. We track the status and creation date of these cases on the Case table.

Uploaded diagnostic images are stored in Image entities associated with particular cases. Images have metadata, such as file path, labels and other properties being used for training our model. If your system uses a machine learning model, then the Models entity will have information about them, e.g. type (YOLO, DETR), descriptions etc. and performance metrics. This Projects_Models table links these models to projects, allowing for a many to many relationship such that one project may be linked to many models, and likewise one model can be linked to many projects.

The system also keeps historical operational data in the Logs entity, which was used to record case performance; activities on cases include label modifications as well as approvals. Traceability and accountability within the system are ensured. The last thing is the Feedback entity that grabs feedback from doctors on the particular projects to help refine the datasets and to make the model performance better. Combining these different interconnected tables together, it presents a strong basis for managing the many different operations and data flow structures in the MediScan system.

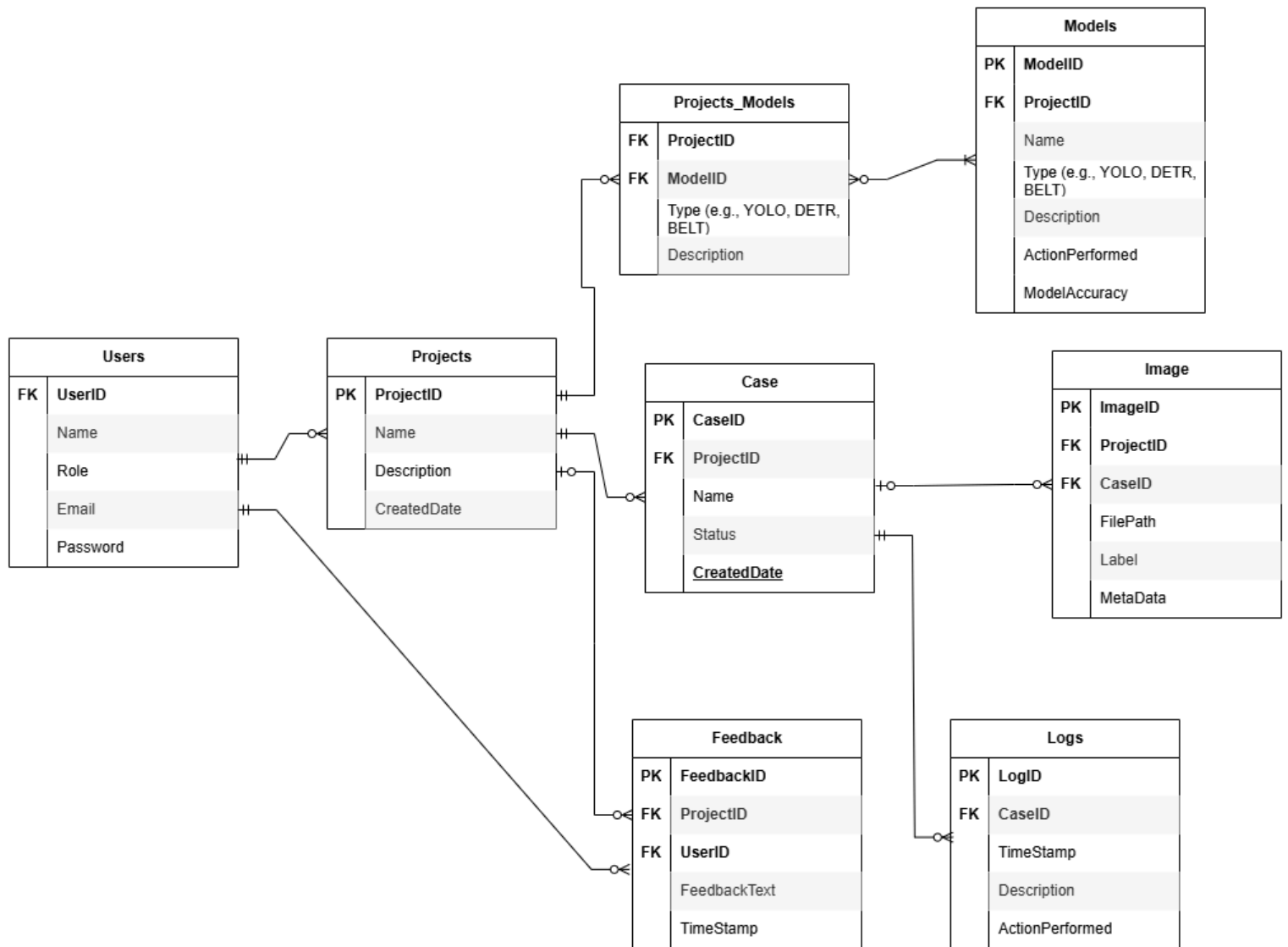


Figure 4.5: Entity Relationship Diagram

4.7 Machine Learning Methodology

4.7.1 Data Collection

- Doctors provide the extracted medical information as images and the system labels them within the system.
- In addition one could also perform additional training importing for example HAM10000 public datasets.

4.7.2 Data Preparation

- Preprocessing such as resizing, normalizing, augmenting the image are these techniques.
- Clinical variables, such as patient age and lesion type are mapped down into a form that fits with machine learning pipelines.

4.7.3 Model Training

- YOLOv8 can all be trained by the system.
- Then, the fine tuning algorithm uses frameworks like TensorFlow or PyTorch, fine tuning each model fine tuned on labeled datasets.
- Model performance is logged in training logs as well as hyperparameters.

4.7.4 Model Optimization

- Through grid search and random search methods, hyperparameter tuning is performed.
- Optimization parameters like learning rate, batch size and number of epochs are things you have to learn.

4.7.5 Model Evaluation

- Models are evaluated using metrics such as:
 - Accuracy: Percentage of the correct predictions among the total predictions.
 - Precision: The true positive predictions scaled by the number of pairs in the dataset N.
 - Recall: Ability to identify cases that have something to do.
 - F1 Score: Mean of precision and recall.

4.7.6 Deployment

- For predictions, models are deployed locally.
- This grants traceability, as well as rollback if necessary.

4.8 Security Measures

To ensure the safety and confidentiality of sensitive medical data, the MediScan system employs several security measures:

- **Local Data Storage:** It's all stored on one secure local machine. This is a cloud agnostic system that means it does not require cloud storage only making its vulnerabilities external to itself.
- **Access Control via VPN:** An access and act as a client to the system is guaranteed via Virtual Private Network (VPN), so the users are connected. In this, people in the secure network are permitted to access the data and models.
- **Role-Based Authentication:** Doctors, patients or system administrators authenticate users by using secure login credentials. With role based permission, the user has access permissions which give roles to the user to interact with functionality for the role.
- **Data Encryption:** All data encrypted in the database, such as user credentials, logs and image metadata are stored this is the reason. This also offers another layer of security keeping it from being accessed by someone that shouldn't.
- **Audit Logs:** Everything is logged through the entire process — image uploads, training of models, predictions — comprehensive logs. They have logs, so you never have to question whose hand something came off of, and if they're even legitimizing it the right way. Logs enable traceability and accountability thus easily catching and responding to the possibility of a breach of security.

CHAPTER 5

EVALUATION AND RESULTS

This chapter includes the extensive testing processes executed to evaluate the precision of the model, the overall functionality of the system, and the comparative duration analysis of verification processes between traditional methods and our technological solution.

5.1 Model Testing

In this section, we undertake a systematic appraisal of the machine learning models. Our evaluation parameters center around the precision, recall, and F1 scores, ensuring that model predictions consistently echo the ground-truth benchmarks.

5.1.1 Objective and Model Testing

The primary objectives of our testing strategy are multifold:

- **Performance Benchmarking** : The model's performance will be evaluated using various metrics such as accuracy, precision, recall, and F1 score. These metrics will be compared against established medical benchmarks to assess the model's proficiency in making clinical predictions. This process will also help identify areas for improvement to enhance the model's diagnostic accuracy.
- **Robustness Assessment** : To ensure the model's reliability, it will be tested for robustness against variations in medical image quality, patient demographics, and the presence of occlusions or anomalies in the images. It is essential for the model to perform consistently across diverse medical conditions and imaging scenarios, ensuring its suitability for real-world clinical applications.

- **Scalability Evaluation:** The model will undergo scalability testing to verify its performance when handling large datasets sourced from various hospitals and clinics. It will be assessed on its ability to process images with different resolutions, formats, and varying levels of quality. Ensuring stable performance across different image qualities and sources is critical for the system's long-term use in clinical environments.
- **Error Analysis:** A comprehensive error analysis will be conducted to identify common patterns in the model's misclassifications. By understanding the conditions under which these errors occur, we will be able to refine the model to minimize these issues and improve its reliability and accuracy for clinical use.

By addressing these objectives, our testing efforts aim to deliver a model that not only demonstrates high performance in controlled clinical settings but also maintains this standard in the dynamic and often unpredictable

5.1.2 Testing Methodology

Description of the Overall Approach to Testing

The testing methodology for our dress code verification system followed a structured approach to validate key functionalities, including model deployment, evaluation, and prediction. Each component was tested individually to ensure proper system integration and performance. The deployment test confirmed that the model could be loaded and accessed through the API without any issues. During the evaluation process, the system responded successfully, but an internal error related to data handling was identified, which needs to be addressed to enable accurate result calculation. In the prediction test, the API functioned as expected; however, no detections were returned in the response, suggesting that the model may require further improvement or that the input data may not have included recognizable features. This testing approach allowed us to assess both the system's technical reliability and practical performance, providing useful insights for future enhancements.

5.1.3 Definition of Success Criteria

For the successful deployment of our dress code verification model, the following criteria have been established:

- **High F1 Score:** The harmonic mean of precision and recall should be high, indicating a balanced and accurate model. A high F1 score signifies the model's strength in accurately identifying dress code violations while minimizing false positives and negatives.
- **Precision:** The model should demonstrate high precision across various Intersections over Union (IoU) thresholds. This precision underscores the model's ability to correctly label positive (violative) instances, which is crucial for maintaining the integrity of the dress code policy.
- **Recall:** The model is expected to maintain high recall rates across different detection scenarios, ensuring it consistently identifies all actual positives. This metric is vital for comprehensive enforcement of dress code regulations.

5.1.4 Model Evaluation

Training process

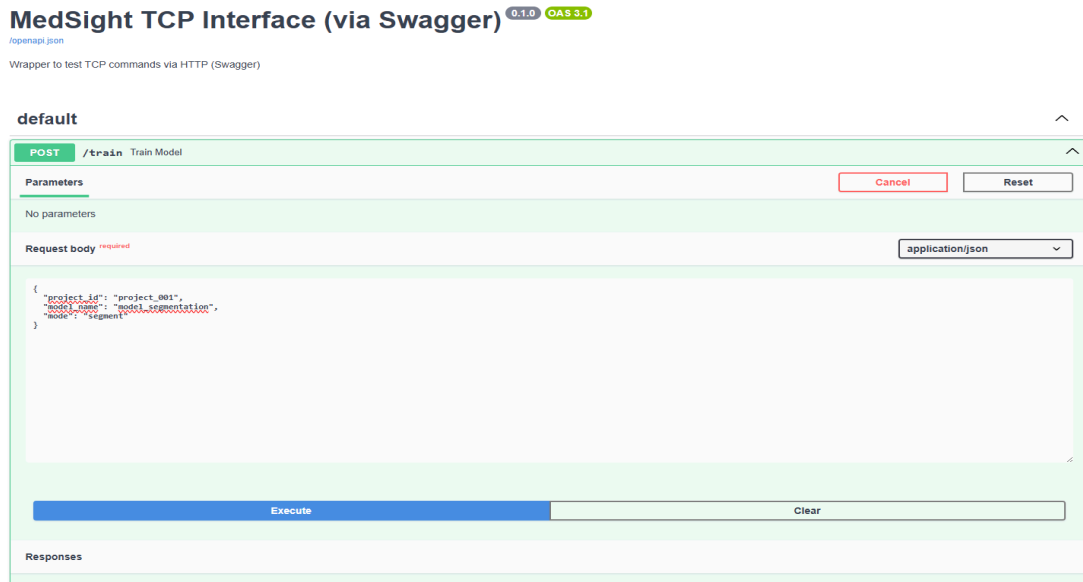


Figure 5.1

Model Training Phase and API Response

This section (*Figure5.1*) explains how the model for the skin cancer segmentation system was trained. It covers the API used to start the training process, the response received from the system, and a simple explanation of the main performance metrics used to evaluate the model.

API Endpoint Overview

The system uses Swagger as the user interface, allowing users to interact with the backend by sending requests over a TCP server. All request and response data are formatted in JSON for consistency and interoperability.

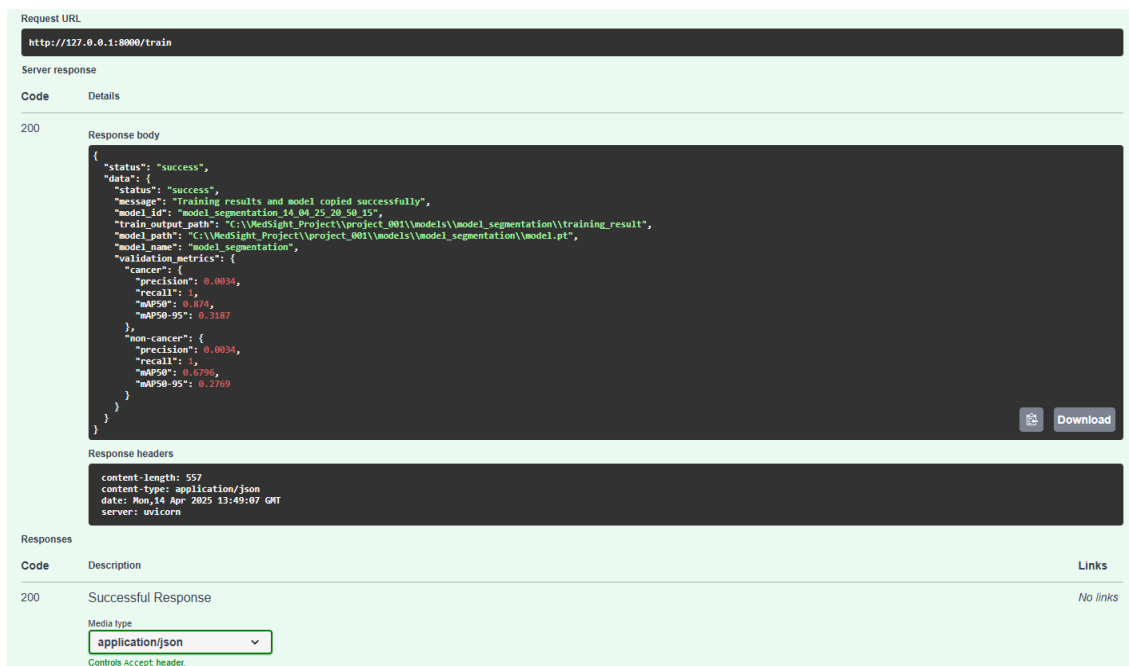


Figure 5.2

Response Summary

After the training request was processed, the API returned a JSON response containing details about the training outcome. The important fields included in the response are as follows:

Field	Description
status	Indicates the success of the operation ("success")
message	Confirms that the training and model saving were completed
model_id	A unique identifier for the trained model based on the date and time

model_name	The name of the trained model (<code>model_segmentation</code>)
train_output_path	The path where the training results were saved
model_path	The path to the final saved model file in <code>.pt</code> format

Results (Figure 5.2)

After training, the model was evaluated using validation data for two classes: **cancer** and **non-cancer**. The metrics used to assess performance are:

Metric	Cancer	Non-Cancer
Precision	0.0034	0.0034
Recall	1.0000	1.0000
mAP@0.50	0.8740	0.6796
mAP@0.50–0.95	0.3187	0.2769

Interpretation

- **Precision** is very low (0.0034) for both classes, indicating a high number of false positives.
- **Recall** is perfect (1.0), meaning the model was able to detect all cancerous and non-cancerous regions in the validation data.
- **mAP@0.50** is relatively high, especially for the cancer class (0.874), suggesting the model can identify and localize cancer regions accurately at a basic threshold.
- **mAP@0.50–0.95** values are lower, showing that the model's accuracy decreases when stricter localization criteria are applied.

Evaluation protocol

POST /evaluate Evaluate Model

Parameters

No parameters

Request body ^{required}

application/json

```
{
  "project_id": "project_001",
  "model_name": "model_segmentation",
  "mode": "segment",
  "eval_type": "test"
}
```

Execute Clear

Responses

Figure 5.3

Model Evaluation

The performance of the segmentation model "**model_segmentation**" under project ID "**project_001**" was evaluated using a POST request to the /evaluate endpoint. The evaluation was carried out in test mode, which means that a different dataset was used to test the model's efficacy. The request was successfully processed over the API interface and submitted in JSON format.

Responses

Curl

```
curl -X 'POST' \
  'http://127.0.0.1:8000/evaluate' \
  -H 'accept: application/json' \
  -H 'Content-Type: application/json' \
  -d '{
    "project_id": "project_001",
    "model_name": "model_segmentation",
    "mode": "segment",
    "eval_type": "test"
  }'
```

Request URL

http://127.0.0.1:8000/evaluate

Server response

Code	Details
200	Response body <pre>{ "status": "success", "data": { "error": "Evaluation failed: 'list' object has no attribute 'mean'" } }</pre> Response headers <pre>content-length: 96 content-type: application/json date: Mon, 14 Apr 2020 11:50:36 GMT server: uvicorn</pre>

Responses

Code	Description	Links
200	Successful Response	No links

Media type

application/json

Controls Accept header.

Figure 5.4

Evaluation Result

Testing of the evaluation API involved a POST request to verify the segmentation model's operation. The server returned an HTTP status code of 200 OK, meaning the request had been successfully received and processed at the communication level.

However, the actual evaluation process encountered an error. The response returned the following message: **"Evaluation failed: 'list' object has no attribute 'mean'"**

According to this error, the backend code attempted to use a Python list to execute the `mean()` function. This method is not supported by native Python lists, so the operation failed. This can be fixed by converting the data to a NumPy array, which supports the `mean()` function, or by computing the average using a common formula such as `sum(list) / len(list)`.

Despite the successful API execution, a logical issue was identified in the evaluation process. The problem is currently under investigation to ensure accurate and reliable model evaluation in future.

Deployment process

POST /deploy Deploy Model

Parameters

No parameters

Request body required

application/json

```
{
  "project_id": "project_001",
  "model_name": "model_segmentation"
}
```

Execute

Clear

Responses

Figure 5.5

Responses

Curl

```
curl -X 'POST' \
  'http://127.0.0.1:8000/deploy' \
  -H 'accept: application/json' \
  -H 'content-type: application/json' \
  -d '{
    "project_id": "project_001",
    "model_name": "model_segmentation"
  }'
```

Request URL

http://127.0.0.1:8000/deploy

Server response

Code

Details

200

Response body

```
{
  "status": "success",
  "message": "Model 'model_segmentation' deployed successfully"
}
```

Download

Response headers

```
content-length: 81
content-type: application/json
date: Mon, 14 Apr 2025 14:07:38 GMT
server: uvicorn
```

Responses

Code	Description	Links
200	Successful Response	No links

Media type

application/json

Controls Accept header.

Figure 5.6

Deployment Result

A POST request was sent to the /deploy API with the model name model_segmentation. The server responded with a 200 OK status and the message, "Model model_segmentation deployed successfully." This confirms that the model was deployed correctly and is ready for further processes such as evaluation or prediction.

Prediction process

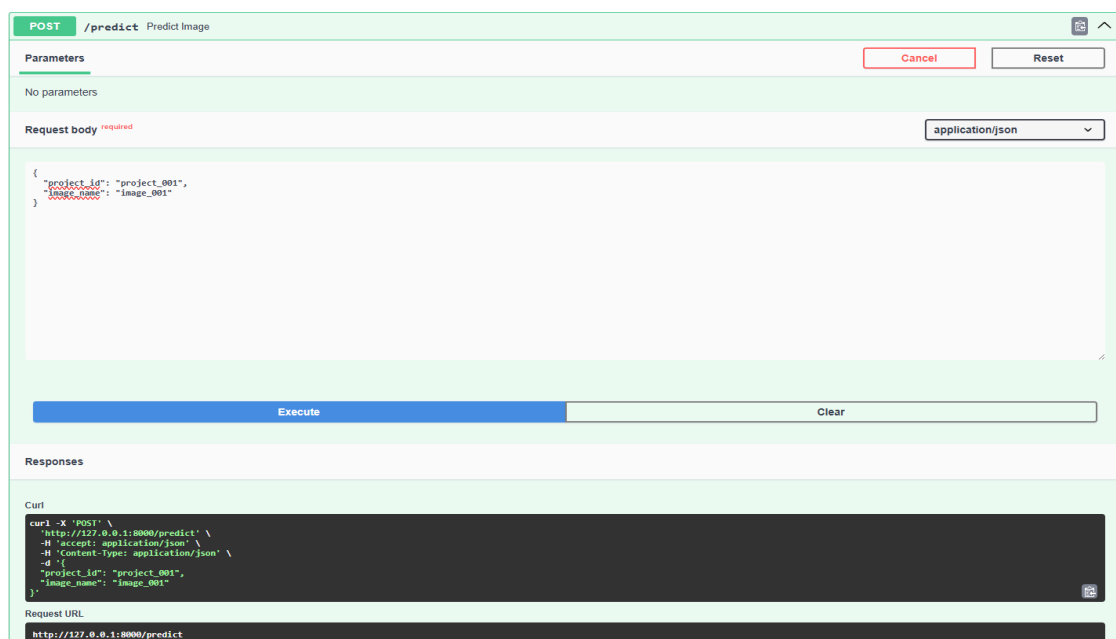


Figure 5.7

Model Prediction

To classify images, a POST request was made to the /predict endpoint. The picture name (**image_001**) and project ID (**project_001**) are included in the input. The model analyzes the image in response to this request and returns a prediction result, such as whether the image shows cancer or not. The request was successfully completed using the Swagger UI and is organized in JSON format.

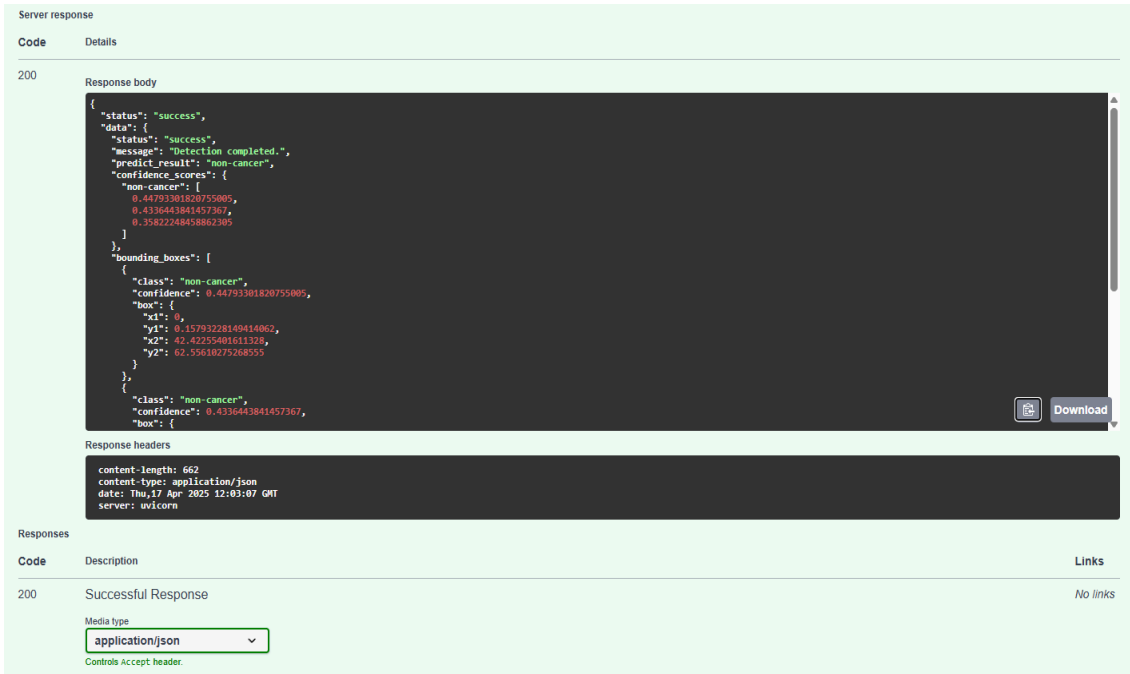


Figure 5.8

Prediction Result

The model successfully completed the prediction and classified the input dermoscopic image as non-cancer, indicating that no malignant skin lesions were detected. A high level of confidence in this classification is supported by multiple confidence scores (0.44, 0.43, 0.35) generated by the model. In addition, the system produced bounding boxes that localize specific regions of interest within the image that were analyzed. Each bounding box is associated with the "non-cancer" class and includes coordinate values (x1, y1, x2, y2) along with a corresponding confidence level. This output integrates both image classification and object detection, which enhances the interpretability and diagnostic accuracy of the skin cancer detection process.

5.1.5 Unit testing

The backend and API have been tested using the pytest module from the Python library. The tested protocols are the following: training, evaluate, deploy, predict, dashboard and connection closing. The result is all passed with 4 warnings that are not affecting the performance of the system.

```
===== warnings summary =====
medsight\Lib\site-packages\pydantic\_internal\_fields.py:132
D:\Code\Project\MediScan\medsight\Lib\site-packages\pydantic\_internal\_fields.py:132: UserWarning: Field "model_name" in TrainRequest has conflict with protected namespace "model_".

You may be able to resolve this warning by setting 'model_config['protected_namespaces'] = ()'.
warnings.warn(

medsight\Lib\site-packages\pydantic\_internal\_fields.py:132
D:\Code\Project\MediScan\medsight\Lib\site-packages\pydantic\_internal\_fields.py:132: UserWarning: Field "model_name" in EvaluateRequest has conflict with protected namespace "model_".

You may be able to resolve this warning by setting 'model_config['protected_namespaces'] = ()'.
warnings.warn(

medsight\Lib\site-packages\pydantic\_internal\_fields.py:132
D:\Code\Project\MediScan\medsight\Lib\site-packages\pydantic\_internal\_fields.py:132: UserWarning: Field "model_name" in DeployRequest has conflict with protected namespace "model_".

You may be able to resolve this warning by setting 'model_config['protected_namespaces'] = ()'.
warnings.warn(

test_swagger_api.py::test_dashboard
D:\Code\Project\MediScan\swagger_api.py:146: UserWarning: Starting a Matplotlib GUI outside of the main thread will likely fail.
plt.figure(figsize=(10, 6))

-- Docs: https://docs.pytest.org/en/stable/how-to/capture-warnings.html
===== 6 passed, 4 warnings in 6.20s =====
```

5.1.6 User story

Doctor User Stories

1. Model Management

- *As a doctor, I want to create and manage machine learning models so that I can customize them for my specific medical diagnosis tasks.*

2. Data Labeling

- *As a doctor, I want to label medical images in real-time so that the system can improve model accuracy through supervised learning.*

3. Model Evaluation

- *As a doctor, I want to evaluate model performance using metrics like precision and recall so that I can choose the most reliable model for deployment.*

4. Project Creation

- *As a doctor, I want to create diagnostic projects with relevant metadata so that I can organize different clinical studies or tasks.*

5. Image Approval

- *As a doctor, I want to review and approve image labels before they are used for training so that data quality is maintained.*

6. Model Deployment

- *As a doctor, I want to deploy trained models for real-time diagnostics so that I can provide immediate analysis for patient images.*

7. Feedback and Refinement

- *As a doctor, I want to provide feedback on prediction results so that the models can be refined and made more accurate.*

Patient User Stories**1. Image Upload**

- *As a patient, I want to upload images related to my skin condition so that the system can provide diagnostic predictions.*

2. View Results

- *As a patient, I want to receive a diagnosis and confidence score from the system so that I can understand whether I need to see a specialist.*

3. Access History

- *As a patient, I want to view my prediction history so that I can track changes over time and share results with healthcare providers.*

CHAPTER 6

CONCLUSION AND Discussion

6.1 Conclusion

The backend and API were found to operate as expected following testing of the system's deployment, evaluation, and prediction features. The evaluation API responded appropriately, and the model was successfully deployed; however, an error occurred due to the use of a function that is incompatible with standard Python lists. To enable accurate result calculation, this needs to be fixed. The system handled the request in the prediction test without any issues, but no objects were found, and the bounding boxes and confidence scores were both blank. This implies that the model may require additional work or that the input may not have included detectable objects. The system is functioning properly overall, but it could perform better if the evaluation and prediction procedures were improved.

6.2 Discussion

6.2.1 Future work

This project successfully developed a skin cancer detection system using machine learning techniques to analyze images of skin lesions. While the system achieved promising results, there are several areas where future improvements can be made to enhance its accuracy, reliability, and practical usability.

6.2.2 Expand the Dataset size

The current model's capacity to manage variability in the real world is limited by the small dataset it was trained on. Future research should incorporate a bigger and more varied collection of skin cancer images to enhance its robustness and performance. This ought to include:

- People with different skin tones and age groups.

- A broader range of skin cancer types, as well as benign (non-cancerous) conditions.
- Images taken in various lighting conditions and from different angles.

6.2.3 Improve Model Performance

While the current model performs reasonably well, future work could explore more advanced techniques, such as:

- Using different models for testing, training, and deployment, such as DETR and BEIT.
- Implementing transfer learning with pre-trained models on medical image datasets.

6.2.4 Limitations

1. **Imbalanced Model Performance:** The model exhibits low precision, indicating a tendency to generate false positives, which could undermine the reliability of diagnostic results, despite achieving high recall.
2. **Lack of Diversity and Size in the Dataset:** The model's generalizability was limited because it was trained on a small dataset that did not represent a range of skin tones, age groups, and environmental conditions.
3. **Inconsistent Prediction Outcomes:** On occasion, the model was unable to generate reliable detections, indicating that it needed more training or improvement.
4. **Dependence on Manual Annotation:** The annotation process relies on medical professionals, which can be time-consuming and introduces the possibility of human error or bias.
5. **Lack of Graphical Interfaces (GUI):** The existing system lacks an intuitive front-end interface, restricting its usability for non-technical users, including clinicians.
6. **Limited Deployment in Real-World Environments:** The system has not yet been tested in real-time clinical settings, and its scalability and performance in such environments remain unverified.

6.2.5 Ethical Considerations

1. **Data Privacy and Security:** Medical data must be handled in compliance with privacy regulations. Although local storage and VPNs are used, formal data protection protocols are necessary.
 2. **Model Training Bias:** Diagnostic fairness among various groups may be impacted by biased results from a lack of diversity in the dataset.
 3. **Lack of Interpretability:** The decision-making process of the model is difficult to understand, which could erode clinician confidence and prevent clinical adoption.
 4. **Accountability for Misdiagnosis:** The system does not define who is responsible in the case of diagnostic errors, which is critical for ethical and legal clarity.
-

REFERENCES

1. Chen, J., et al. "MLflow: An Open Platform to Manage the ML Lifecycle."
2. TensorFlow Extended Documentation, "Model Analysis and Deployment." Available at: TensorFlow.org.
3. Kubeflow Documentation, "A Machine Learning Toolkit for Kubernetes." Available at: Kubeflow.org.
4. Dawson, M., & Sato, T. "A Process for Evaluating Machine Learning Models for Healthcare Applications." ResearchGate, 2021.
5. Redmon, J., et al. "You Only Look Once: Unified, Real-Time Object Detection." CVPR, 2016.
6. Bochkovskiy, A., et al. "YOLOv4: Optimal Speed and Accuracy of Object Detection." arXiv, 2020.
7. YOLOv8 Documentation, "Real-Time Object Detection for Medical Imaging." Available at: YOLOv8.com.
8. Carion, N., et al. "End-to-End Object Detection with Transformers." ECCV, 2020.
9. EfficientDet Research Team, "Scalable and Efficient Object Detection." arXiv, 2021.
10. Litjens, G., et al. "A Survey on Deep Learning in Medical Image Analysis." Medical Image Analysis Journal, 2017.
11. Tschandl, P., et al. "The HAM10000 Dataset: A Large Collection of Multi-Source Dermatoscopic Images." Nature Scientific Data, 2018.
12. Esteva, A., et al. "Dermatologist-Level Classification of Skin Cancer with Deep Neural Networks." Nature, 2017.

APPENDIX A

The Role of Machine Learning in Medical Diagnostics

It is true that ML has defined healthcare in a completely different way; systems can now analyze unthinkable complicated medical data and extract patterns beyond the reach of human understanding. ML models have the most impact in medical imaging in Parkinson's disease detection, skin cancer, cardiovascular conditions and malignancies.

In fact, ML has been extensively used in the early diagnosis field. However, early detection has been found to greatly improve patient outcomes (and, therefore, overall outcomes) because detection early gives to earlier treatments and less sophisticated disease. ML models can identify such anomalies in dermatoscopic images and accurately diagnose them, and the human eye may miss out on several subtle anomalies in these images.

It's been said that integrating ML in healthcare is tough. So they have to deal with that sensitive data carefully, correctly and reliably. Finally, medical ML systems are used to interpret and analyze the data coming from the evolving real world so need to be constantly monitored to the normal. To respond to these challenges, MediScan has created a system able to meet the medical practitioners needs, combining thoroughness and regulatory compliance.